NEMO chip: SIPO, PISO and Y-Flash modes explained

Oct 12, 2026 · @Rahaf

1. Big picture

Dima's SIPO and PISO code is the logic that sits **inside the NEMO chip**, not code for our FPGA. Our FPGA controller has to be its mirror: a serializer that feeds the chip's SIPO, and a deserializer that reads the chip's PISO.

- **Evidence it is chip-side:** `piso.sv` instantiates Tower 180 nm standard cells (`ANTENNA_18_SVT_WTN`, `DLY3_X4_18_SVT_WTN`) that do not exist on a Xilinx FPGA. The chip schematic `top_before_pads` (image_4.png) has exactly these ports: `clk_sipo`, `sipo_in`, `cs`, `rst_sipo`, `clk_piso`, `shift`, `rst_piso`, `sipo_out`, `piso_out`.
- **What the chip does:** SIPO in, Y-Flash array computes, LVR / VTC / TDC / DSA digitize, PISO out (spec block diagram, slide 5).
- **What the FPGA does:** drives every blue digital input with clean, correctly ordered edges, and samples the two serial outputs.
- **Bonus use of Dima's code:** it is a ready-made behavioural model of the chip. We can put it in our Vivado testbench as the DUT and check our FPGA logic against it before touching silicon (the Tower cells need small simulation stubs).

2. The chip's digital pins (blue rows)

The blue rows in `IO-Pins-Dima` are the chip's **digital** pins: 22 inputs the FPGA must drive and 4 outputs it must read, so 26 FMC pins in total (we have 32 verified). Names follow the latest pad ring (rows 188 to 204) and the `top_before_pads` schematic; the earlier pin list calls `s0_p` .. `s2_p` simply `s0` .. `s2`. Purple rows (`clk`, `pulse`, `dft0/1`) are "RF" pads but are still digital, so they are included. Orange rows are analog biases and supplies (`v_read`, `v_write`, `v_erase`, `vth`, `vcch_wl` ...) that come from lab instruments, not the FPGA.

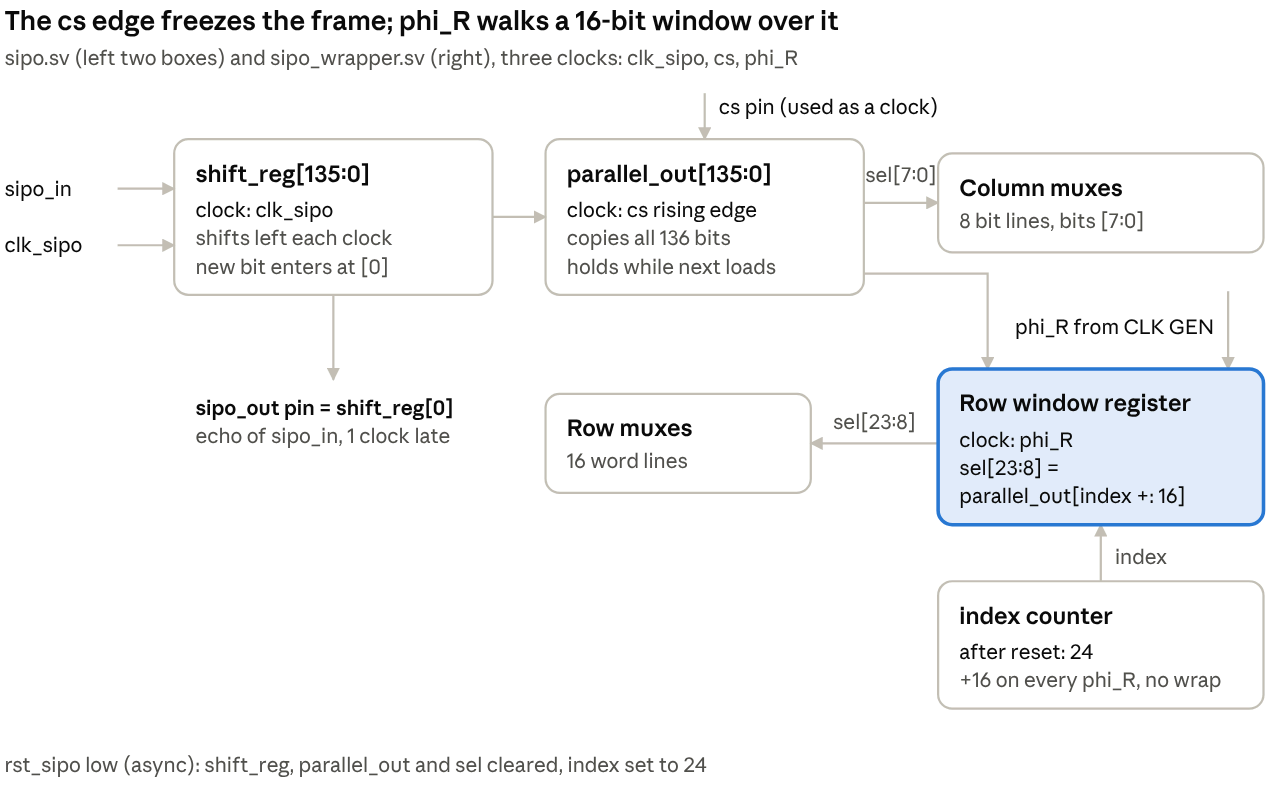
Pin	Dir (chip)	Block	Meaning	Source
`clk_sipo`	in	SIPO	Shift clock: one rising edge per bit	Dima's RTL (`clk`)
`sipo_in`	in	SIPO	Serial data, sampled on rising `clk_sipo`	Dima's RTL
`cs`	in	SIPO	**Edge**, not a level: rising edge copies the shift register into `parallel_out`	Dima's RTL (`clk_s`)
`rst_sipo`	in	SIPO	Async reset, active low in the RTL	Dima's RTL (`rst_n`)
`sipo_out`	out	SIPO	`shift_reg[0]`: echo of `sipo_in`, one clock late	Dima's RTL
`clk_piso`	in	PISO	Load / shift clock	Dima's RTL
`shift`	in	PISO	0 = load parallel data on next edge, 1 = shift	Dima's RTL (`load`)
`rst_piso`	in	PISO	Async reset, active low in the RTL	Dima's RTL
`piso_out`	out	PISO	Serial result, MSB first	Dima's RTL
`clk`	in	Core	Main clock into CLK GEN, which makes phi_R, phi_1, phi_2	Block diagram
`pulse`	in	Core	Probably the program / erase pulse timing	**Guess, confirm**
`rst_dsa`	in	Core	Clears the DSA (shift-and-add) accumulators	Name + diagram
`reset`, `set`	in	Core	Not documented	**Ask Dima**
`s0_p`, `s1_p`, `s2_p`	in	Core	3 bits, likely operation / mode select	**Guess, confirm**
`s0_dft` .. `s3_dft`	in	DFT	Which internal signal goes to the DFT outputs	Name + DFT sheet

Pin	Dir (chip)	Block	Meaning	Source
`b0_dft` .. `b2_dft`	in	DFT	3 bits = which of the 8 columns is observed	Name + "x8"
`dft0_buff`, `dft1_buff`	out	DFT	Buffered observation outputs (DFT<1:0>)	Schematic

The stand-alone test structure (`reset_sa` , `set_sa` , `clk_sa` , `input_sa` , `s0/1_dft_sa` , `piso_out_sa`) also appears in the pin list but is not in the latest pad ring; worth asking whether it is still on the chip.

3. SIPO: 136 bits in, 24 select lines out

The SIPO receives a 136-bit frame, latches it on a `cs` edge, and then presents it to the array 24 bits at a time: 8 fixed column bits plus 16 row bits that step forward on every phi_R edge.



SIPO internals · 3 registers, 3 clocks

Three registers, three clocks: we own `clk_sipo` and `cs`, the chip owns phi_R. Because `cs` is a clock, a glitch on that wire latches garbage, so the FPGA should drive it from a register, never from logic.

`sipo.sv`, two registers in two clock domains

- `WIDTH = BL_num + 8*WL_num = 8 + 8*16 = 136`.
- Shift register on `clk_sipo`: `shift_reg <= {shift_reg[134:0], serial_in}`. Shifts left, new bit enters at bit 0. After 136 clocks the **first bit sent sits in bit 135**, the last bit sent in bit 0.
- Output register on `cs` (port `clk_s`): `parallel_out <= shift_reg`. Only the rising edge of `cs` updates the outputs, so the array never sees bits while they are still shifting.
- `sipo_out = shift_reg[0]`: the bit that just came in. It is a one-clock-late echo of `sipo_in`, useful as an "is the chip alive" loopback, not a chain output.

`sipo_wrapper.sv`, stepping through the frame with phi_R

- `sel[7:0] = parallel_out[7:0]`: always connected, goes to the column multiplexers (SEL<7:0>, 8 bit lines).
- `sel[23:8]` is a 16-bit window into `parallel_out` that moves 16 bits per rising edge of `phirst` (phi_R, made inside the chip by CLK GEN): `sel[23:8] <= parallel_out[index +: 16]; index <= index + 16;`, starting from `index = 24` after reset. It goes to the row multiplexers (SEL<23:8>, 16 word lines).
- Meaning: the 128 row bits are 8 slices of 16 bits. Slice k holds bit k of every input X_0..X_15. The array is fed one bit-plane per phi_R cycle, which is how 8-bit inputs become time pulses on the word lines (the X_0[7]..X_0[0] pulses in the block diagram).

Frame layout (`parallel_out` bits)

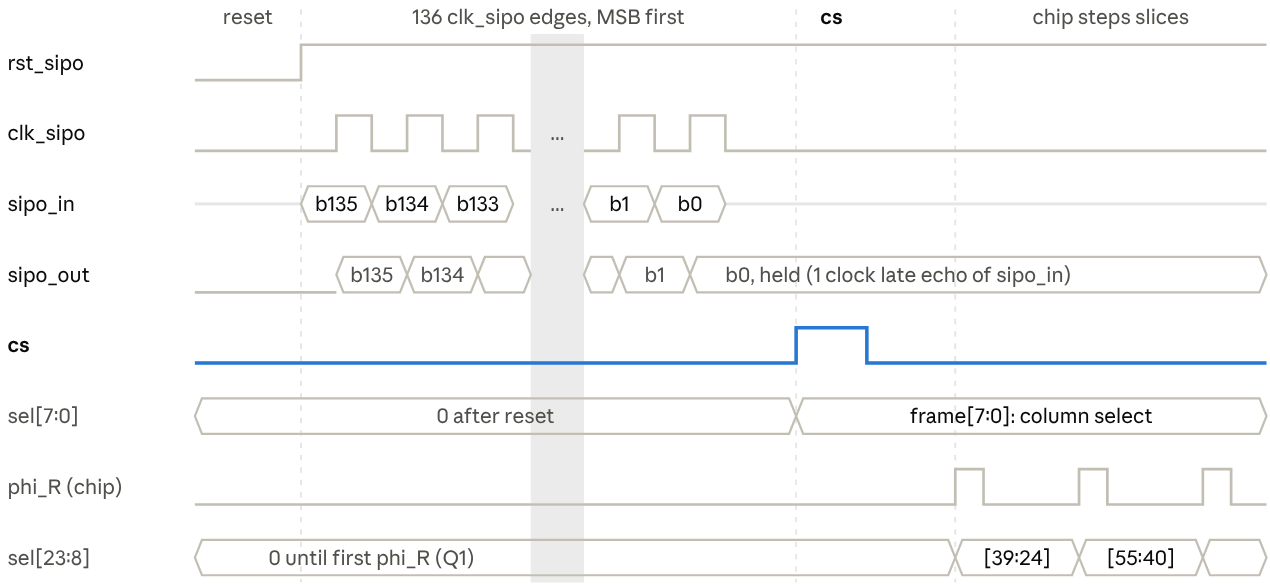
Bits	Count	Goes to	Content
[7:0]	8	Column muxes, always	Column / bit-line select
[23:8]	16	Row muxes, at reset	Slice 0 (see open question Q1)
[39:24]	16	Row muxes, after 1st phi_R	Slice 1
…	…	…	One slice per phi_R
[135:120]	16	Row muxes, after 7th phi_R	Slice 7

What the FPGA must do (SIPO driver)

1. Pull `rst_sipo` low, then release it (clears both registers and sets `index = 24`).
2. Send the 136 bits **MSB first** (bit 135 first, bit 0 last) on `sipo_in`. Change `sipo_in` on the falling edge of `clk_sipo` so it is stable at the rising edge.
3. Give exactly 136 rising edges on `clk_sipo`, then stop the clock low.
4. Give one clean rising edge on `cs`. The 24 `sel` lines now hold the frame.
5. Optional check: compare `sipo_out` with `sipo_in` delayed by one clock; a mismatch means a wiring or level problem.
6. phi_R comes from the chip, so the FPGA only provides `clk`; it does not step `sel` itself.

Load the SIPO: 136 clocks MSB first, then one cs edge

Not to scale; the break hides 131 identical clock periods



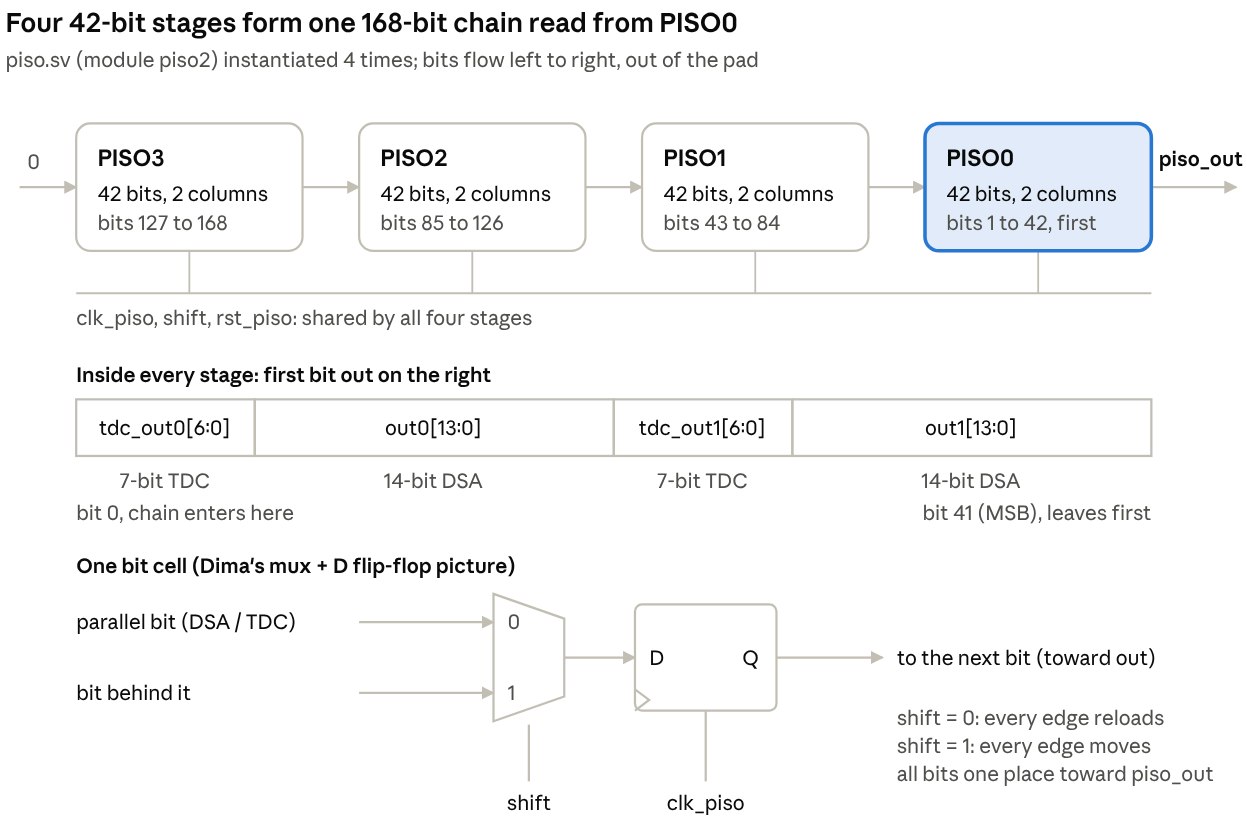
Change sipo_in on the falling clk_sipo edge; raise cs only after the 136th edge, with clk_sipo parked low.

SIPO load timing · FPGA drives rst_sipo, clk_sipo, sipo_in, cs

The top five rows are pins (four we drive, `sipo_out` we read); the bottom three live inside the chip and are shown so you can see when the array actually gets the bits.

4. PISO: 8 column results out on one wire, 168 bits

Four 42-bit PISO stages are chained into one 168-bit shift register; the FPGA loads it with one clock while `shift = 0`, then clocks it out MSB first on `piso_out`.



PISO chain · 4 stages x 42 bits = 168 bits

For the FPGA the whole chain behaves like one 168-bit shift register: we never address a stage, we just count bits. Which physical column sits in which stage is still open (Q5).

One stage (`piso2` , 42 bits)

- Each stage holds the results of **two columns**: a 14-bit DSA result (`out0` , `out1`) and a 7-bit TDC code (`tdc_out0` , `tdc_out1`) per column. 4 stages x 2 columns = the 8 columns (the "x8" and "x4" in the block diagram).
- Packing, first bit out to last: `out1[13:0]` , `tdc_out1[6:0]` , `out0[13:0]` , `tdc_out0[6:0]` .
- `load` is the pad `shift` . On each rising `clk_piso` :
  - `shift = 0` : **load**, `stage <= {out1, tdc_out1, out0, tdc_out0}` (it reloads on every edge while low).
  - `shift = 1` : **shift left**, the MSB leaves, `prev_serial_bit` enters at the LSB.
- `serial_out` is `stage[41]` through 4 delay cells (`DLY3`). They only add hold margin inside the chip, a few ns, irrelevant at our MHz rates.

- This is exactly Dima's mux + D-flip-flop picture: the mux is the `if (!load)`, the flip-flops are `stage`. In the picture the first mux takes a constant 1; in the RTL the chain end gets `1'b0`.

The chain

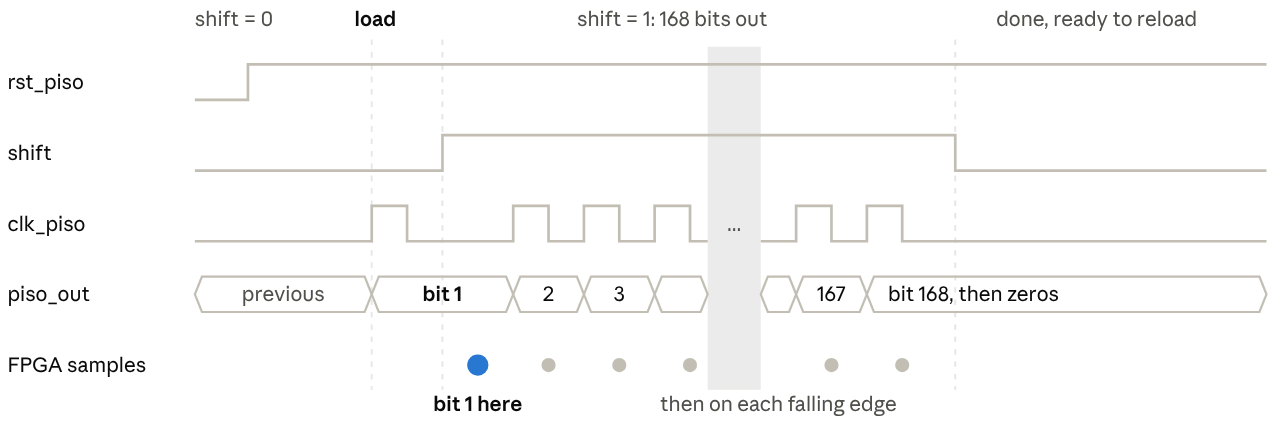
PISO3 feeds PISO2, PISO2 feeds PISO1, PISO1 feeds PISO0, and the pad `piso_out` is PISO0's output. So the stream on the wire is PISO0's 42 bits, then PISO1's, PISO2's, PISO3's, then zeros.

What the FPGA must do (PISO reader)

1. Release `rst_piso` (active low in the RTL).
2. Wait until the chip's computation is finished (DSA done), with `shift = 0`.
3. Give **one** rising edge on `clk_piso`: all four stages capture.
4. Set `shift = 1` while `clk_piso` is low.
5. **Read bit 1 now**, before any more clocks: the MSB is already on `piso_out` right after the load edge. Forgetting this loses the first bit and shifts every field by one.
6. Then 167 more rising edges; sample `piso_out` on each falling edge (half a period after the shift). 168 bits total.
7. Split the 168 bits as 4 x [14 + 7 + 14 + 7].

Read the PISO: one load edge, then read bit 1 before shifting

Not to scale; the break hides 162 identical shift periods



1 load edge + 167 shift edges = 168 bits. Skip the early read and every field slips by one bit.

PISO read timing · FPGA drives rst_piso, shift, clk_piso

The dots are where our reader latches `piso_out`. Sampling on the falling edge gives half a period of margin on both sides, which also absorbs the chip's output delay cells and the pad.

5. The Y-Flash cell and its modes

A Y-Flash cell is two NMOS transistors sharing one floating gate (FG); the charge on the FG sets its threshold Vth, and Vth is the stored weight. Program raises Vth, erase lowers it, read measures the current that Vth allows (Danial et al., Nature Electronics 2020; Wang, Y-Flash modelling slides 2021).

The cell

- Terminals: common **drain D**, **read source SR**, **injection source SI**, substrate (always 0 V). No gate pin: the FG is set mainly through its coupling to the drain (C_GD is the largest capacitance).
- **Injection transistor:** short channel, built to inject charge; used for program and erase.
- **Read transistor:** built to resist injection; used for reading so reads do not disturb the weight.
- Standard Tower 180 nm CMOS, single poly, no extra masks.

Read: subthreshold "memristor" mode

A small read voltage V_r on the drain keeps the cell in subthreshold, where the current is exponential in Vth:

```
I_{DS} \approx I_{read}\, e^{\frac{C_R V_{DS}}{m V_T}}\, e^{-\frac{V_{th}}{m V_T}}
```

With Vth between 1 and 2 V the current spans 1 nA to 1 uA, roughly 145 kOhm to 145 MOhm. About 65 distinct levels are usable (about 6 bits). SI floats or is tied to the drain during read.

Mode	Bias (paper values)	Physics	Effect on Vth	Effect on current	Array granularity
Read	Small V_r on D, sources at 0	Subthreshold conduction	None	Measured	Whole array at once
Program (RESET)	About 5 V on D (4.5 V in arrays), sources 0 V	Channel hot electrons injected into FG	Up	Down (weight weaker)	Per row / per cell

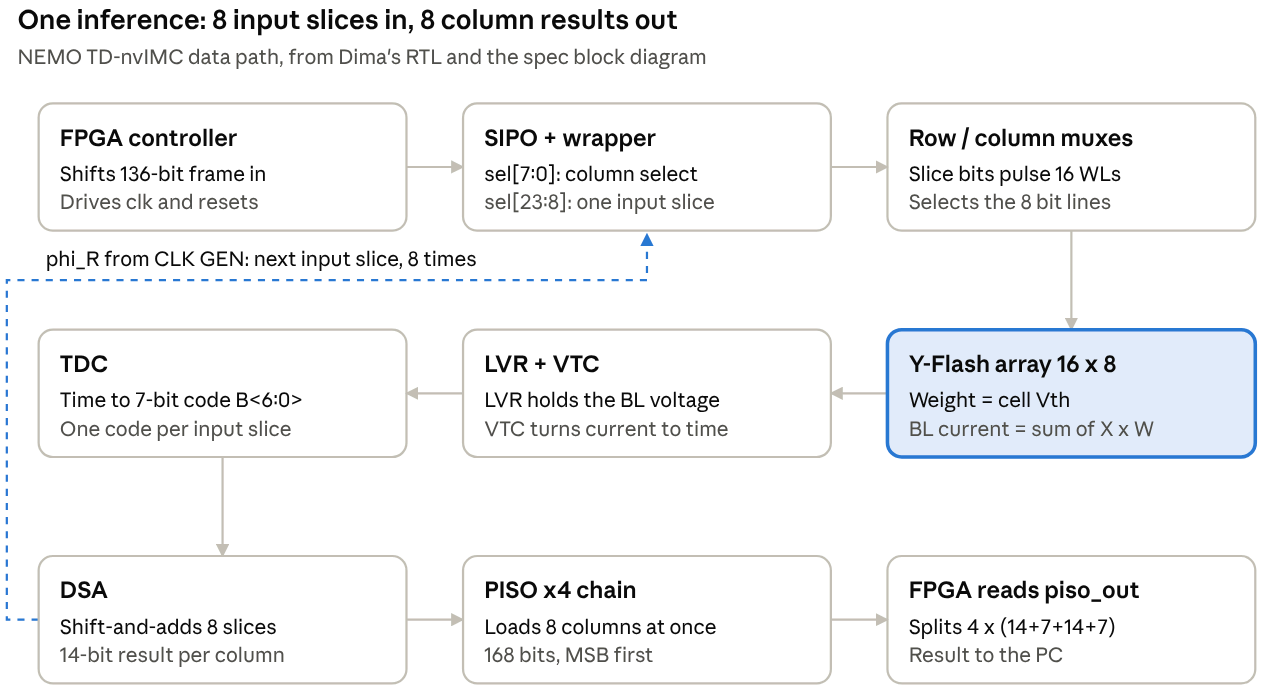
Mode	Bias (paper values)	Physics	Effect on Vth	Effect on current	Array granularity
Erase (SET)	About 8 V (7.5 to 8 V) on SI, D and substrate 0 V	Band-to-band hot holes into FG	Down	Up (weight stronger)	Whole column segment
Compute (VMM)	Input pulses on word lines, V_r on bit lines	Ohm's law per cell, Kirchhoff sum per column	None	Column current = sum of X times W	All cells in parallel

Points that matter for the controller

- Weights are tuned with **short repeated pulses**: amplitude and **duration** set the step. Reaching a target needs a program-verify loop: read, compare, pulse, read again (1% accuracy in the paper).
- A small drain voltage (about 1 V) on a column blocks erase there, which is how unselected cells are protected.
- On our chip the high voltages come from bias pins (`v_write` , `v_erase` , `v_read` , `vcch_wl` , `vcch_bl`) driven by lab supplies. The FPGA's role is the **timing and selection**: which row and column (SIPO `sel`), which mode (likely `s0_p` .. `s2_p`), and how long (likely `pulse`).

6. How one inference runs through the chip

An inference is 8 passes through the array, one per input bit-slice, stepped by phi_R; the DSA adds the 8 partial results with the right binary weights, and only then does the PISO read out.



inference data path · 9 stages, 1 loop per input slice

The TDC gives 7 bits per slice, and shift-adding 8 slices grows that to the 14-bit `out` word, which is why the PISO carries both `tdc_out` (7 bits) and `out` (14 bits) per column. The TDC-per-slice and DSA reading is inferred from the block diagram and bit widths; confirm with Dima.

7. Open questions for Dima

Reading the RTL line by line turned up two likely bugs and several unknowns; Q1 and Q2 change how we build the frame, so they come first.

1. **Q1, slice 0 may never reach the rows.** In `sipo_wrapper`, the reset branch loads `sel[23:8] <= parallel_out[23:8]`, but `parallel_out` is reset by the same `rst_n`, so it loads zeros. The first phi_R then jumps to bits [39:24]. Bits [23:8] are never shown. Is that intended (FPGA should put slice 0 at [39:24]?) or a bug?
2. **Q2, the window runs off the end.** `index` never wraps. After the 7th phi_R it reads [135:120]; the 8th reads [151:136], past `WIDTH = 136` (undefined). How many phi_R cycles per inference, and must we pulse `rst_sipo` and reload the frame each time?
3. **Q3, bit and slice order.** Is slice 0 the input MSB or LSB? Which `sel` bit is which row? What exactly do the 8 bits of `sel[7:0]` select?

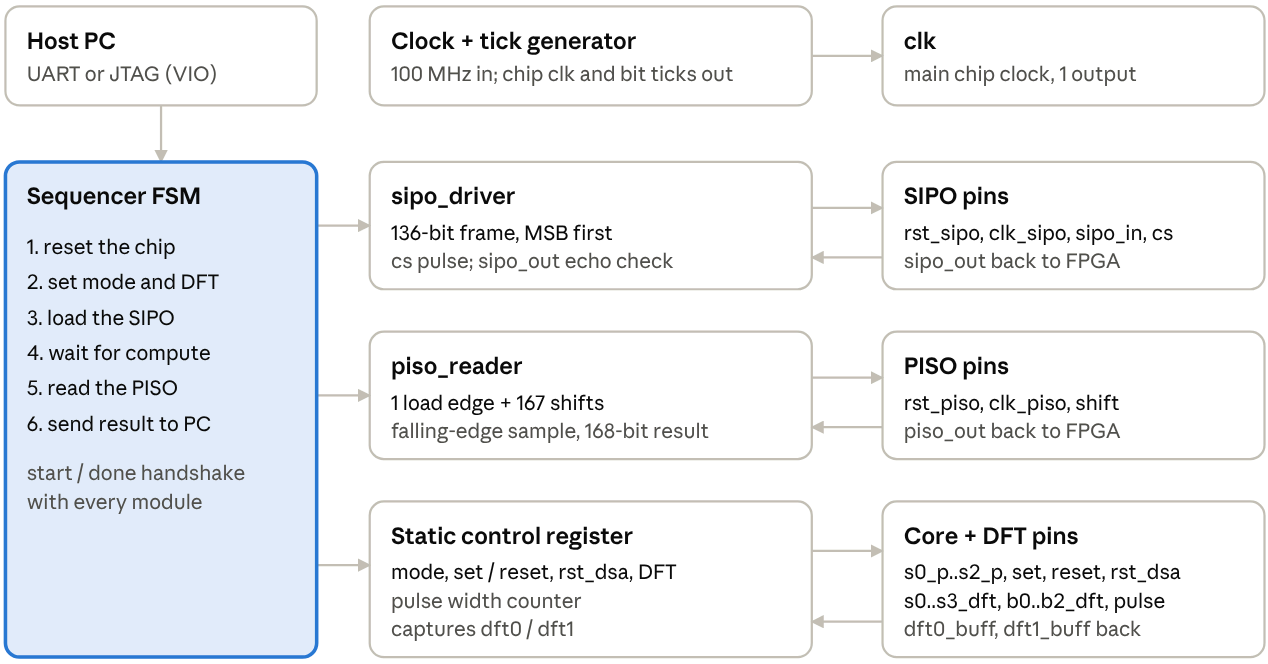
4. **Q4, phases.** The pad list has `clk` but no phi_1, phi_2, phi_R pins, and CLK GEN is on chip. Do we only supply `clk` (what frequency?), or do we need to generate the non-overlapping phases ourselves, as the PDR slide assumed?
5. **Q5, PISO column mapping.** Which physical column is `out0` / `out1` of which PISO stage? Pin sheet says `piso_in<13:0>` while the block diagram says `Dout<14:0>`: 14 or 15 bits?
6. **Q6, unknown pins.** Meaning and polarity of `reset`, `set`, `s0_p` .. `s2_p`, `pulse`, and the DFT selects. Are `rst_sipo` / `rst_piso` active low at the pad, as in the RTL?
7. **Q7, levels and speed.** Digital pad voltage (our FMC is 1.8 V LVCMOS18), and the maximum `clk_sipo` / `clk_piso` rate.
8. **Q8, our slide 3 wording.** Weights live in the Y-Flash cells and are written with program / erase pulses; the SIPO carries the **inputs and the row/column selects**, not the weights themselves. Is that the right reading?

8. Proposed FPGA controller

Split the controller so each module owns one group of chip pins and knows nothing about the others; one sequencer decides the order. Then the open questions only touch one module each: Q1 to Q3 change the frame inside `sipo_driver`, Q5 only changes how we split the 168 bits, Q4 only touches the clock generator.

One sequencer drives four small modules, each owning its own chip pins

Proposed FPGA side (ZedBoard PL); left: control, middle: modules we write, right: chip pins



Arrows to the right: the FPGA drives the pin. Arrows to the left: the chip drives it, through a 2-FF synchronizer.
Pin budget: 22 outputs + 4 inputs, inside the 32 verified LA_P pins on J2 / J3.

proposed FPGA controller · 1 sequencer, 4 modules, 26 pins

Ways to make life easier

- **Run all chip-facing logic from one slow tick** (for example 1 MHz from the 100 MHz clock), not from divided clocks. Every pin changes on a tick in the 100 MHz domain, so timing is easy to reason about and easy to slow down for the multimeter.
- **Register every output pin** (output flip-flop, `IOB = TRUE` in the XDC), above all `cs` and the two shift clocks: a glitch there latches or shifts garbage.
- **Make the frame length, read length and bit order parameters** (`SIPO_BITS = 136`, `PISO_BITS = 168`, `MSB_FIRST = 1`), so Dima's answers change one line.
- **Self-test before the chip arrives:** jumper `sipo_in` to a spare input and check the echo; feed the PISO reader from a second FPGA shift register that imitates the chip.
- **Simulate against Dima's RTL first:** our modules + his three files in one Vivado testbench gives a cycle-exact check of both sequences above.
- **Put an ILA on all 26 pins:** we see on screen exactly what the chip saw.

9. Next steps

We can start building now and keep the parts that depend on Dima's answers behind parameters.

- [ ] Ask Dima Q1 to Q8 (section 7) at tomorrow's meeting.
- [ ] Copy `sipo.sv`, `sipo_wrapper.sv`, `piso.sv` into a chip model for simulation, with stub modules for `ANTENNA_18_SVT_WTN` and `DLY3_X4_18_SVT_WTN`.
- [ ] Write `sipo_driver.sv` on the FPGA: 136-bit frame register, MSB-first shifter, `clk_sipo` from the 100 MHz divider, one `cs` pulse at the end, `sipo_out` echo check.
- [ ] Write `piso_reader.sv`: one load edge, read bit 1 before clocking, 167 shifts, sample on falling edge, 168-bit result register.
- [ ] Testbench: FPGA driver + reader against Dima's model, a known frame in and known `out0` .. `tdc_out1` values out.
- [ ] New pin map: 22 FPGA outputs + 4 inputs on the verified LA_P pins, with an XDC (the current 16-out / 16-in loopback split does not fit).
- [ ] Hardware check with no chip: loop `sipo_in` back to an input and confirm the frame on the ILA.